

데이터 파이프라인 설계 및 구축

1. 왜 v2를 만들었는가

v1에서 실제 운영 데이터를 조사하면서 한계를 확인했습니다. 일부 데이터가 제한되어 있거나 서로 연결되지 않아 사용자 행동을 지속적으로 분석하기 어려웠습니다.

v2는 서비스가 계속 운영되며 데이터가 쌓이는 상황을 가정했습니다. 필요한 데이터를 일관되게 적재하고 같은 분석을 반복할 수 있는 구조가 목표였습니다.

실제 서비스 데이터는 현재 발생한 사용자 행동을 확인하는 데 사용하고 합성 데이터는 데이터가 충분히 쌓인 상황을 가정해 파이프라인과 분석 구조를 검증하는 데 활용했습니다.

Raw → Staging → Mart로 데이터를 단계별로 관리하고 적재와 가공 과정은 Airflow로 자동화했습니다.

2. 실제 데이터와 합성 데이터를 함께 사용한 이유

실제 데이터

실제 서비스에서 발생한 데이터는 Supabase PostgreSQL에 저장하고 BigQuery Raw로 적재했습니다.

```
Next.js / Vercel
  ↓
Supabase PostgreSQL
  ↓
BigQuery Raw
```

현재 실제 데이터 규모는 다음과 같습니다.

- 사용자 23명
- 질문 24개
- 투표 700건

현재 데이터만으로는 장기간 리텐션이나 다양한 사용자 행동을 분석하기에 규모가 부족했습니다.

합성 데이터

실제 서비스와 같은 구조로 합성 데이터를 생성했습니다. 실제 데이터가 충분히 쌓이지 않은 상황에서도 파이프라인과 분석 구조를 검증하려는 목적이었습니다.

generator/synthetic-v2.yaml

↓

generate.py

↓

CSV 40개

7.4GB

↓

Local PostgreSQL

↓

BigQuery Raw

생성한 데이터는 다음과 같습니다.

- 사용자 20,000명
- 질문 240개
- 140M+ rows

실제 데이터와 동일한 40개 테이블과 컬럼 구조를 사용했습니다. 그래서 두 데이터에 같은 적재 과정과 분석 쿼리를 적용할 수 있습니다.

합성 데이터는 실제 서비스와 분리된 Local PostgreSQL에서 관리했습니다. 또한 CSV를 바로 BigQuery에 적재하지 않고 Local PostgreSQL에서 먼저 검증했습니다.

17개의 SQL 정합성 검사를 수행하고 FK / UNIQUE / CHECK 제약조건을 적용했습니다. 데이터가 분석 단계로 넘어가기 전에 오류를 먼저 확인합니다.

3. BigQuery Raw: 실제 데이터와 합성 데이터를 함께 관리

검증이 끝난 실제 데이터와 합성 데이터는 BigQuery의 동일한 Raw 영역에 적재했습니다.

데이터 출처와 적재 시점을 구분하기 위해 `_source`와 `_loaded_at`을 추가했습니다.

`_source`

- supabase : 실제 데이터
- local : 합성 데이터

`_loaded_at`

- BigQuery에 적재된 시각

실제 데이터와 합성 데이터는 서로 다른 데이터지만 ID가 겹칠 수 있습니다.

예를 들어 실제 질문 ID와 합성 질문 ID가 모두 8일 수 있습니다.

그래서 Staging에서 데이터를 연결할 때는 ID뿐 아니라 데이터 출처도 함께 사용했습니다.

```
ON a.id = b.id
AND a._source = b._source
```

두 조건이 모두 일치하는 경우에만 데이터를 연결했습니다.

4. 증분 적재: 변경된 데이터만 가져오기

서비스가 운영되는 동안 데이터는 계속 늘어납니다. 매번 전체 데이터를 다시 적재하지 않고 새로 추가되거나 수정된 데이터만 가져오게 구성했습니다.

각 테이블의 마지막 적재 시점은 `_load_state`에 기록하고 다음 적재에서는 해당 시점 이후 변경된 데이터만 가져왔습니다.

데이터가 증가해도 적재되는 것은 필요한 데이터뿐입니다.

5. Staging: 분석에 필요한 기준 정리

Raw에 원본 데이터를 저장했다면, Staging에서는 분석할 때 반복해서 사용하는 조건과 조인 기준을 정리했습니다.

`CREATE OR REPLACE VIEW`를 사용해 원본 데이터를 수정하지 않고 View 형태로 관리했습니다. 처음 6개로 시작해 필요한 데이터를 추가하면서 최종적으로 10개 View를 구성했습니다.

```
stg_user
stg_question
stg_vote
stg_vote_session
stg_vote_received
stg_friend_edge
stg_report
...
```

친구 관계

친구 관계는 양쪽 사용자 기준으로 각각 저장됩니다. 그대로 집계하면 하나의 관계가 두 번 계산될 수 있었습니다.

`stg_friend_edge`에서 `user_low_id < user_high_id` 기준을 적용해 하나의 관계를 한 번만 집계했습니다.

투표자 정보

`voter_id`는 일반 사용자 분석과 분리했습니다.

누가 나에게 투표했는지는 하트 구매로 확인할 수 있는 정보입니다. 그래서 별도 View로 관리하고 대시보드에서는 사용하지 않았습니다.

매출 기준

MVP의 충전 데이터에는 테스트 거래가 섞여 있어 실제 매출과 구분할 필요가 있었습니다.

Staging에서 다음 기준을 정리했습니다.

```
is_stub  
is_success  
is_revenue
```

성공한 거래 중 테스트 거래가 아닌 경우만 실제 매출로 집계합니다.

6. Mart: 대시보드에서 사용하는 데이터

Staging에서 분석 기준을 정리한 뒤 대시보드에서 바로 사용하는 형태로 Mart를 구성했습니다.

Mart는 최종 8개입니다.

```
mart_user  
mart_user_activity  
mart_daily  
mart_funnel_step  
mart_distribution  
mart_report  
mart_heart_flow  
mart_backlog_weekly
```

처음에는 월별 집계를 중심으로 만들었지만 대시보드에서 주간 필터와 전주 비교가 필요해지면서 구조를 변경했습니다.

- 모든 Mart에 날짜 기준 포함
- 비율과 함께 분자·분모 저장
- 필요한 경우 원본 수준에서 다시 집계할 수 있도록 구성
- 퍼널과 분포는 이벤트 날짜 또는 코호트 날짜 기준으로 집계

조회 비용 감소

합성 데이터에서 가장 큰 vote_candidate 테이블은 약 4.9 GiB였습니다.

대시보드에서 같은 데이터를 반복해서 조회하면 비용이 커질 수 있어 자주 사용하는 데이터는 Mart에서 미리 집계했습니다.

2026-08-18 기준 Mart는 약 495 MiB였습니다.

7. Airflow: 데이터 처리 과정 자동화

Raw → Staging → Mart 과정을 데이터가 추가될 때마다 직접 실행하지 않도록 Airflow로 자동화했습니다.

전체 과정은 두 개의 DAG로 나누었습니다.

① Raw 데이터 적재

ping_raw_load

```
├─ extract_load.py
└─ verify_load.py
```

↓

Raw 적재 및 검증

↓

② 분석용 데이터 생성

ping_mart_build

```
├─ quality_check.py
└─ build.py --layer mart
```

↓

Mart 생성

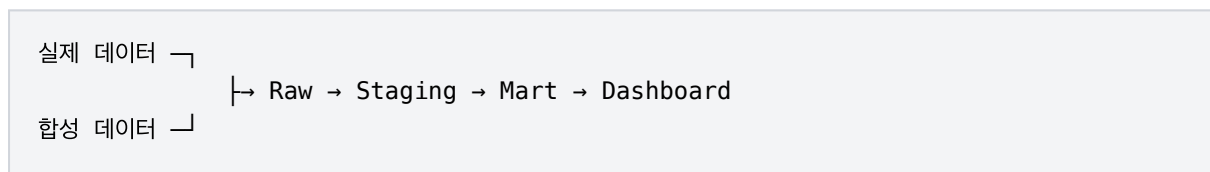
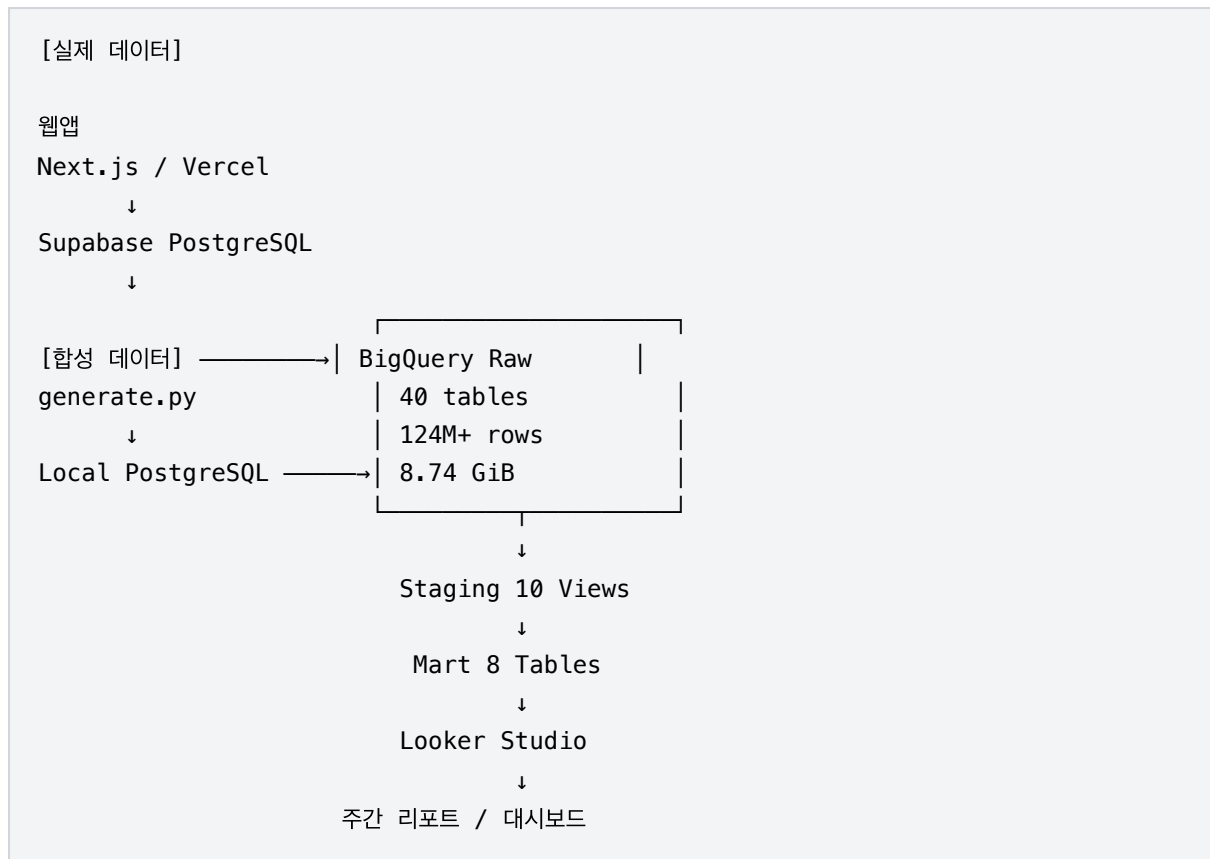
첫 번째 DAG에서 Raw 데이터를 적재하고 품질을 확인한 뒤 검증을 통과한 경우에만 두 번째 DAG에서 Mart를 생성합니다.

Raw 적재나 품질 검사가 실패하면 기존 Mart를 그대로 둡니다.

데이터 적재부터 검증, 분석용 데이터 생성까지 자동으로 실행됩니다.

8. 최종 구조

앞에서 구성한 데이터 구조와 자동화 과정을 연결하면 다음과 같습니다.



v1에서는 실제 운영 데이터를 분석하면서 서비스의 이용 현황과 이탈 패턴을 확인했지만 데이터가 제한되어 있어 사용자 행동을 연결하거나 같은 분석을 반복하는 데 한계가 있었습니다.

v2에서는 이러한 한계를 바탕으로 실제 데이터와 합성 데이터를 같은 구조로 관리하고 데이터 적재부터 분석용 데이터 생성까지의 과정을 자동화했습니다.

새로운 데이터가 추가되더라도 기존 구조를 활용해 같은 분석을 반복하고 이전 데이터와 비교할 수 있습니다. 여기까지가 v2에서 구축한 데이터 기반입니다.

향후 실제 데이터가 충분히 쌓이면 사용자 세그먼트별 행동 차이를 비교하고, v1에서 확인하기 어려웠던 이탈 원인을 실제 데이터로 검증할 계획입니다.